PROJECT IMPLEMENTATION PLAN & TASK ARRANGEMENT

Intelligent ERP Data Navigator

Custom-Trained Autonomous Text-to-SQL AI Agent for AdventureWorks2019 with Egyptian Arabic NLP & Modern Web UI

Repository: github.com/.../Text-To-SQL-AI-Agent-

Created: October 2026

Version: 1.0

Target DB: SQL Server 2019 (`AdventureWorks2019`)

Base Model: `Qwen2.5-Coder-3B`

Status: Active — Dev

1. Team Roster & Balanced Engineering Squads

To guarantee **100% technical equity** and deep hands-on coding experience for every team member, the project is organized into **three core Pair-Engineering Squads**. No member is restricted to documentation or testing alone; instead, every engineer builds core production modules (AI/NLP, Backend Agent, Full-Stack UI, and Automated Testing) and co-authors the technical documentation for the components they build.

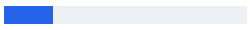
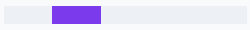
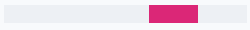
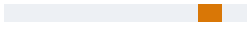
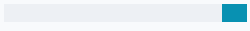
#	Engineer	Core Technical Role (Squad)	Cross-Phase Rotation & Shared Ownership
1	**Moustafa Maher**	Lead Architect & Full-Stack / Agent Engineer	Co-builds Self-Correction Agent, WebSocket Engine & System Architecture Docs
2	**Adel Ahmed**	AI Fine-Tuning & LLM Inference Engineer	Co-builds QLoRA Training Pipeline, GGUF Engine, Prompt Templates & Training Docs
3	**Ro'aa Ashraf**	Data/NLP Engineer & RAG Schema Architect	Co-builds Synthetic Dataset Generator, RAG Schema Pruner, Schema API & API Docs
4	**Abdallah Sameh**	Backend Core & Database Security Engineer	Co-builds FastAPI Core, SQL Executor, AST Security Validator & DB Benchmarks
5	**Renad Hakim**	NLP Normalizer, Analytics & Security Engineer	Co-builds Egyptian Arabic Normalizer, Chart Recommender, Answer Synthesizer & QA Suite
6	**Retaj Hakim**	Frontend Lead & Data Visualization Engineer	Co-builds Next.js 14 RTL App, Chat UI, Recharts Suite, SQL Inspector & UI Manual

Collaborative Pair-Programming & Rotation Model

- **Squad A — AI, Dataset & RAG Pipeline (Adel Ahmed + Ro'aa Ashraf):** Collaborate side-by-side in Phase 1 & 2 to build the 71-table schema extractor, synthetic Egyptian Arabic dataset generator, QLoRA fine-tuning pipeline, model evaluation suite, and RAG schema pruner.
- **Squad B — Backend Core, NLP & Agent Intelligence (Abdallah Sameh + Renad Hakim + Moustafa Maher):** Pair up in Phase 2 & 4 to engineer the Egyptian Arabic normalizer, AST SQL safety gate, pyodbc execution engine, 3x self-correction reflection loop, answer synthesizer, chart recommender, and security test suite.
- **Squad C — Next.js RTL Frontend, Visualizations & Streaming (Retaj Hakim + Moustafa Maher + Ro'aa Ashraf):** Collaborate in Phase 3 & 4 to build the Next.js 14 RTL interface, interactive Schema Explorer, Recharts analytics, SQL Inspector, and live WebSocket streaming hooks.
- **Shared Documentation & QA Responsibility:** Every engineer writes unit tests and technical documentation for their own code modules, and all 6 members co-present the final system in Phase 5.

2. Master Timeline (10-Week Collaborative Roadmap)

Each phase is co-executed by paired engineers working in parallel, rotating across AI training, backend orchestration, frontend engineering, and system verification.

Timeline	Phase	Core Technical Milestone	Co-Leading Engineers	Schedule (W1–W10)
Week 1–2	**Phase 0**	Infrastructure, DB Setup, GPU Env & Next.js Init	All 6 Engineers (Paired)	
Week 3–4	**Phase 1**	Synthetic Dataset Gen, QLoRA Training & GGUF	Adel + Ro'aa + Renad	
Week 5–6	**Phase 2**	FastAPI Core, RAG, AST Gate, NLP & Reflection Loop	Abdallah + Renad + Adel + Moustafa	
Week 7–8	**Phase 3**	Next.js RTL UI, Charts, Schema Explorer & State	Retaj + Ro'aa + Moustafa	
Week 9	**Phase 4**	E2E Wiring, WebSockets, Security Audit & Latency	Renad + Moustafa + Abdallah	
Week 10	**Phase 5**	Shared Docs, Benchmarks, Demo Video & Defense	All 6 Engineers (Equal Split)	

3. Acceptance Criteria & Target Architecture

Project Acceptance Criteria (Definition of Done)

The system is considered production-ready when all benchmark metrics below pass verification:

Criterion	Target Benchmark
Execution Accuracy (EX)	$\geq 85\%$ on 200+ validation dataset
Valid SQL Rate (VSR)	$\geq 95\%$ syntactically valid T-SQL
Safety Compliance (SC)	100% — Zero DML/DDL reaches DB
End-to-End Latency	< 3.0 seconds per query (RTX 3050)
UI RTL Rendering	Perfect Right-to-Left layout with Cairo font
Self-Correction Loop	Automatic reflection retry up to 3 times
Technical Workload Parity	10 technical tasks per member (60 total)
Shared Documentation	Co-authored README, API Ref, QA & Slides

Target Repository Structure

```
Text-To-SQL-AI-Agent-/
|-- DataSet/ (AdventureWorks2019.bak)
|-- Docs/ (Co-authored by all 6 members)
| |-- schema_metadata.json (via script)
| |-- System-Design.md & API-Reference.md
| `-- QA-Report.md & User-Manual.md
|-- training/ (Adel + Ro'aa + Renad)
| |-- dataset/ (extract_schema,
generate_data)
| |-- train_lora.py & convert_to_gguf.py
| `-- evaluate_model.py
|-- models/ (qwen2.5-coder-
3b.Q4_K_M.gguf)
|-- backend/ (Abdallah + Moustafa + Renad
+ Ro'aa)
| |-- main.py, config.py,
requirements.txt
| |-- tests/ (validator, normalizer,
routes)
| |-- agent/ (model_runner, normalizer,
| | prompts, schema_loader, validator,
| | executor, self_correction, synth,
chart)
| `-- api/routes/ (chat, health, schema)
|-- frontend/ (Retaj + Moustafa + Ro'aa)
| `-- src/ (app/, components/, hooks/,
lib/)
`-- .gitignore & README.md
```

4. Detailed Technical Task Assignments by Engineer

1. Moustafa Maher

Lead Architect & Full-Stack / Autonomous Agent Engineer

Technical Ownership (10 Tasks): Co-architects the repository and Python/Node environments, builds the autonomous `SelfCorrector` reflection loop in the backend, implements real-time WebSocket token/event streaming across FastAPI and Next.js, builds the typed Axios API client, and co-authors the System Architecture documentation.

[TASK-M-01 & M-02] ~ GitHub Architecture, Branch Protection, Kanban & `.gitignore` — Phase 0 | Week 1–2

- **TASK-M-01 (Repo & CI Governance):** Configure protected `main` and integration dev branches; set up feature branch workflow and GitHub Project v2 Kanban board with balanced technical labels.
- **TASK-M-02 (Environment & `.gitignore`):** Author root `.gitignore` (blocking `*.gguf`, `*.bak`, `.venv/`, `node_modules/`) and lock dependencies in `backend/requirements.txt` and `training/requirements_training.txt`.
- **Deliverable:** Protected repository, Kanban board, and locked dependency specifications.

[TASK-M-03] ~ Implement Autonomous Self-Correction Agent (`self_correction.py`) — Phase 2 | Week 5–6

- **Technical Pair:** Works with **Adel Ahmed** and **Abdallah Sameh**.
- **Action Plan:** Build `SelfCorrector` in `backend/agent/self_correction.py` orchestrating up to `MAX_RETRIES = 3`. Execute initial SQL generation → AST validation → DB execution. On pyodbc error, dynamically inject the failed T-SQL and SQL Server error message into `build_reflection_prompt()` to autonomously repair the query.
- **Deliverable:** `self_correction.py` with structured retry telemetry and unit tests.

[TASK-M-04] ~ Implement Backend WebSocket Streaming Route (`WS /api/v1/chat/ws`) — Phase 2 | Week 5–6

- **Technical Pair:** Works with **Abdallah Sameh**.
- **Action Plan:** Implement asynchronous WebSocket endpoint in `backend/api/routes/chat.py` that streams live agent execution stages (`thinking`, `schema_pruned`, `sql_generated`, `sql_validated`, `executed`, `answer`) in real time to the frontend client.
- **Deliverable:** Asynchronous WebSocket streaming endpoint verified with Postman/wscat.

[TASK-M-05 & M-06] ~ Build Frontend Typed API Client (`api.ts`) & WebSocket Hook (`useAgentStream.ts`) — Phase 3–4 | Week 7–9

- **Technical Pair:** Works with **Retaj Hakim**.
- **TASK-M-05 (`frontend/src/lib/api.ts`):** Build typed Axios/Fetch client connecting Next.js to FastAPI (`sendMessage`, `getSchema`, `getHealth`) with interceptors and error boundaries.
- **TASK-M-06 (`frontend/src/hooks/useAgentStream.ts`):** Implement custom React hook managing WebSocket connection lifecycle, state transitions, and live stage progression for the chat UI.
- **Deliverable:** Complete frontend-to-backend REST and WebSocket integration layer.

[TASK-M-07 to M-10] ~ Full-Stack Integration, Code Reviews & System Architecture Documentation — Phase 4–5 | Week 9–10

- **TASK-M-07 & M-08:** Lead cross-branch PR code reviews, resolve integration conflicts on dev, and verify end-to-end Docker/local startup scripts.
- **TASK-M-09 & M-10 (Shared Docs & Defense):** Author the System Architecture & Deployment sections of `README.md` and `Docs/System-Design-and-Technical-Plan.md`; present the Architecture & Self-Correction slides in the final defense.
- **Deliverable:** Production release on `main` and architecture documentation.

2. Adel Ahmed

AI Fine-Tuning & LLM Inference Engineer

Technical Ownership (10 Tasks): Leads CUDA/GPU setup, co-builds the synthetic dataset pipeline with Ro'aa, writes the 4-bit QLoRA fine-tuning and GGUF quantization scripts, builds the `llama-cpp-python` inference runner and ChatML prompt engine, and documents the AI training methodology.

[TASK-A-01 & A-02] ~ CUDA 12.1 GPU Setup & Zero-Shot Baseline Benchmarking — Phase 0 | Week 1–2

- **TASK-A-01:** Configure CUDA 12.1, PyTorch 2.4, and `bitsandbytes` on RTX 3050 (plus Google Colab T4 fallback notebook `training/colab_train.ipynb`).
- **TASK-A-02:** Download `Qwen/Qwen2.5-Coder-3B-Instruct` and execute zero-shot Egyptian Arabic SQL generation tests to establish pre-training baseline metrics in `training/baseline_test.txt`.
- **Deliverable:** Verified GPU training environment and baseline benchmark report.

[TASK-A-03] ~ Co-Build Synthetic Egyptian Arabic Dataset Generator (`generate_synthetic_data.py`) — Phase 1 | Week 3–4

- **Technical Pair:** Co-developed with **Ro'aa Ashraf** (Adel builds complex SQL templates: JOINs, CTEs, Window functions, Aggregations; Ro'aa builds schema context injector & dialect phrasing).
- **Action Plan:** Generate $1,000+$ gold T-SQL pairs across `Sales`, `Production`, `Purchasing`, `HumanResources`, and `Person` splits into `train_dataset.json` (800)

and `val_dataset.json` (200).
- **Deliverable:** Validated multi-join T-SQL query templates and training split.

[TASK-A-04 & A-05] ~ Write QLoRA Fine-Tuning Script (`train_lora.py`) & GGUF Quantizer (`convert_to_gguf.py`) **Phase 1 | Week 3–4**

- **TASK-A-04 (`train_lora.py`):** Implement 4-bit NF4 QLoRA training (`r=16`, `lora_alpha=32`, `lr=2e-4`, ChatML formatting) using HuggingFace TRL `SFTTrainer` across all attention and MLP projections.
- **TASK-A-05 (`convert_to_gguf.py`):** Merge LoRA adapters into base FP16 weights (`merge_and_unload()`) and quantize via `llama.cpp` to `q4_K_M` (≈ 1.9 GB) for fast 4GB VRAM inference.
- **Deliverable:** Trained LoRA adapters and quantized `qwen2.5-coder-3b-egyptian-sql.Q4_K_M.gguf`.

[TASK-A-06 & A-07] ~ Implement LLM Inference Engine (`model_runner.py`) & Prompt Engine (`prompt_templates.py`) **Phase 2 | Week 5–6**

- **Technical Pair:** Works with **Moustafa Maher** and **Renad Hakim**.
- **TASK-A-06 (`model_runner.py`):** Build `ModelRunner` wrapping `Llama(n_gpu_layers=35, n_ctx=4096)` with deterministic SQL generation (`temperature=0.1`) and token-streaming support.
- **TASK-A-07 (`prompt_templates.py`):** Engineer ChatML system and user prompt builders for SQL Generation, Error Reflection, and Egyptian Arabic Answer Synthesis.
- **Deliverable:** Production LLM inference engine and prompt template module.

[TASK-A-08 to A-10] ~ Co-Build Evaluation Pipeline, Model Tuning & AI Pipeline Documentation **Phase 4–5 | Week 9–10**

- **TASK-A-08:** Co-run `evaluate_model.py` with **Ro'aa** and **Renad**; fine-tune hyperparameters on any domain falling below 85% Execution Accuracy.
- **TASK-A-09 & A-10 (Shared Docs & Defense):** Document the QLoRA Fine-Tuning & Quantization methodology in `Docs/AI-Training-Pipeline.md` and present the AI Model section during the final defense.
- **Deliverable:** $\geq 85\%$ EX model weights and AI training technical documentation.

3. Ro'aa Ashraf

Data / NLP Engineer, RAG Schema Architect & Full-Stack Contributor

Technical Ownership (10 Tasks): Writes Python automation to extract and enrich the 71-table SQL Server schema catalog, co-builds the synthetic Egyptian Arabic dataset generator and automated model evaluator with Adel, implements the backend RAG Schema Pruner (`schema_loader.py`) and `GET /api/v1/schema` endpoint, and builds the interactive Frontend `SchemaExplorer.tsx` component with Retaj.

[TASK-RO-01] ~ Write Automated SQL Schema Metadata Extractor (`extract_schema.py`) — Phase 0 | Week 1–2

- **Technical Pair:** Works with **Abdallah Sameh**.
- **Action Plan:** Write a Python script querying `INFORMATION_SCHEMA.TABLES`, `COLUMNS`, and `sys.foreign_keys` on `AdventureWorks2019` to programmatically generate `Docs/schema_metadata.json` covering all 71 tables, primary keys, 91 foreign keys, and bilingual (English/Arabic) table descriptions.
- **Deliverable:** `training/dataset/extract_schema.py` and validated `Docs/schema_metadata.json`.

[TASK-RO-02 & RO-03] ~ Co-Build Synthetic Dataset Generator & Automated Model Evaluator — Phase 1 | Week 3–4

- **Technical Pair:** Co-developed with **Adel Ahmed**.
- **TASK-RO-02 (Dialect & Schema Injector in `generate_synthetic_data.py`):** Build the Egyptian Arabic linguistic variation engine (mapping business intents like كام عدد، أكتر، إجمالي، السنة اللي فاتت، زباين، مخزن، موردين) and automated DDL schema context builder for the 1,000-sample dataset.
- **TASK-RO-03 (`training/evaluate_model.py`):** Write the automated evaluation script that runs generated SQL vs. gold SQL against SQL Server to compute Execution Accuracy (EX) and Valid SQL Rate (VSR) into `evaluation_report.json`.
- **Deliverable:** Linguistic generator module, `evaluate_model.py`, and `evaluation_report.json`.

[TASK-RO-04 & RO-05] ~ Implement Backend RAG Schema Pruner (`schema_loader.py`) & `/schema` Route — Phase 2 | Week 5–6

- **Technical Pair:** Works with **Abdallah Sameh** and **Renad Hakim**.
- **TASK-RO-04 (`backend/agent/schema_loader.py`):** Implement the RAG Schema Pruner class (`load_catalog()` and `prune_schema(domains, query)`) that dynamically filters the 71 tables down to the top relevant tables and FK paths ($\leq 2,000$ tokens) per user query.
- **TASK-RO-05 (`backend/api/routes/schema.py`):** Implement `GET /api/v1/schema` FastAPI route with domain filtering and search query parameters, plus unit tests in `backend/tests/test_schema_loader.py`.
- **Deliverable:** Production RAG schema pruner, FastAPI schema endpoint, and passing unit tests.

[TASK-RO-06 & RO-07] ~ Build Frontend Interactive Schema Explorer (`SchemaExplorer.tsx`) & Zustand Slice — Phase 3 | Week 7–8

- **Technical Pair:** Works with **Retaj Hakim**.
- **Action Plan:** Build `frontend/src/components/SchemaExplorer.tsx` fetching `GET /api/v1/schema`. Implement collapsible RTL domain accordions (`Sales`, `Production`, `Purchasing`, `HR`, `Person`), real-time table/column search filter, and click-to-insert table hints into the chat store.
- **Deliverable:** Interactive `SchemaExplorer.tsx` sidebar component integrated into the main layout.

[TASK-RO-08 to RO-10] ~ Co-Author API Reference, Demo Recording & Final Presentation Coordination — Phase 4–5 | Week 9–10

- **Action Plan:** Author `Docs/API-Reference.md` covering REST/WS schemas; coordinate the 11-slide master presentation deck (where each engineer populates their own technical slides); and record the 3–5 minute live system demo video.
- **Deliverable:** `API-Reference.md`, unified slide deck, and narrated demo video.

4. Abdallah Sameh

Backend Core & Database Security Engineer

Technical Ownership (10 Tasks): Restores and configures the SQL Server `AdventureWorks2019` database, initializes the FastAPI application and Pydantic configuration, builds the pyodbc SQL execution engine, implements the `sqlglot` AST security validator gate, builds the `POST /api/v1/chat` and `GET /api/v1/health` endpoints, and co-leads database latency optimization.

[TASK-AB-01 & AB-02] ~ SQL Server Database Setup, FastAPI Initialization & `sql_executor.py` — Phase 0 | Week 1–2

- **TASK-AB-01 (FastAPI & Config):** Create `backend/main.py`, CORS middleware, and `backend/config.py` using Pydantic `BaseSettings` (`.env.example`).
- **TASK-AB-02 (`backend/agent/sql_executor.py`):** Restore `AdventureWorks2019.bak` on SQL Server; implement `SQLExecutor` with pyodbc connection pooling, timeout enforcement (`30s`), row capping (`MAX_ROWS=1000`), and structured error capture.
- **Deliverable:** Running FastAPI core and verified `sql_executor.py` connected to SQL Server.

[TASK-AB-03] ~ Implement AST SQL Security Gate (`sql_validator.py`) — Phase 2 | Week 5–6

- **Technical Pair:** Works with **Renad Hakim**.
- **Action Plan:** Implement `SQLValidator` in `backend/agent/sql_validator.py` using `sqlglot` (`dialect="tsql"`). Parse the full Abstract Syntax Tree to strictly allow `SELECT` and `WITH` CTEs while blocking all DML/DDL AST nodes (`Insert`, `Update`, `Delete`, `Drop`, `Alter`, `Create`, `Truncate`) and dangerous procedures (`xp_cmdshell`, `exec`, `openrowset`).
- **Deliverable:** Production AST validator guaranteeing 100% read-only database safety.

[TASK-AB-04 & AB-05] ~ Implement Main Agent Orchestration Route (POST /chat) & Health Endpoint — **Phase 2 | Week 5–6**

- **Technical Pair:** Works with **Moustafa Maher** and **Renad Hakim**.
- **TASK-AB-04 (POST /api/v1/chat):** Wire the complete 6-stage pipeline in `backend/api/routes/chat.py`: Arabic Normalizer → Domain Detector → RAG Schema Pruner → Self-Correction Loop → Answer Synthesizer → Chart Recommender.
- **TASK-AB-05 (GET /api/v1/health):** Implement live system diagnostics in `backend/api/routes/health.py` checking SQL Server heartbeat (`SELECT 1`), loaded GGUF model status, and RTX 3050 VRAM telemetry via `nvidia-smi`.
- **Deliverable:** Verified `/chat` and `/health` endpoints documented in OpenAPI/Swagger.

[TASK-AB-06 to AB-10] ~ Executor Unit Tests, Query Caching, Demo Fallback & Backend Documentation — **Phase 3–5 | Week 7–10**

- **TASK-AB-06 & AB-07:** Write unit tests for `sql_executor.py` (`test_executor.py`); implement LRU query caching and automatic pyodbc reconnection logic for resilient live demos.
- **TASK-AB-08 to AB-10 (Shared QA, Docs & Defense):** Co-execute latency benchmarks with **Renad**, document the Backend Security & Database Execution layer in `Docs/System-Design-and-Technical-Plan.md`, and present the Backend & AST Security section in Week 10.
- **Deliverable:** Resilient SQL execution layer, caching mechanism, and security documentation.

5. Renad Hakim

NLP Normalizer, Analytics Engine & Security/QA Lead

Technical Ownership (10 Tasks): Builds three core production backend modules—the Egyptian Arabic NLP Normalizer & Domain Classifier (`arabic_normalizer.py`), the Arabic Answer Synthesizer (`answer_synthesizer.py`), and the Automated Chart Recommender Engine (`chart_recommender.py`)—while leading the automated pytest/Jest test suites, adversarial SQL injection testing, and latency benchmarking.

[TASK-RE-01 & RE-02] ~ Initialize Automated Testing Frameworks (`pytest` & `Jest`) & Dialect Corpus

Phase 0–1 | Week 1–4

- **TASK-RE-01:** Set up `pytest`, `pytest-asyncio`, and `httpx` fixtures in `backend/tests/conftest.py`, plus `Jest` and `React Testing Library` in `frontend/`.
- **TASK-RE-02 (Phase 1 Pair with Adel & Ro'aa):** Audit synthetic training pairs in `val_dataset.json` for Egyptian dialect naturalness and Franco-Arabic edge cases, building 20 adversarial write-intent prompts.
- **Deliverable:** Configured backend/frontend test runners and audited validation benchmark set.

[TASK-RE-03] ~ Implement Egyptian Arabic NLP Normalizer & Domain Classifier (`arabic_normalizer.py`)

Phase 2 | Week 5–6

- **Technical Pair:** Works with **Ro'aa Ashraf** and **Abdallah Sameh**.
- **Action Plan:** Build `ArabicNormalizer` in `backend/agent/arabic_normalizer.py`:
  - `normalize(text)`: Unify Alef variants (آ / إ / أ → ا), Teh Marbuta (ة → ه), Alef Maksura (ى → ي), and strip tashkeel.
  - `franco_to_arabic(text)`: Translate Franco business slang (توتال → إجمالي, سيلز → مبيعات, أوردرات → طلبيات, ستوك → مخزون).
  - `detect_domain(text)`: Multi-label keyword classifier mapping Egyptian queries to `Sales`, `Production`, `Purchasing`, `HumanResources`, and `Person`.
- **Deliverable:** Production `arabic_normalizer.py` module + 12+ unit tests in `test_normalizer.py`.

[TASK-RE-04 & RE-05] ~ Implement Answer Synthesizer (`answer_synthesizer.py`) & Chart Recommender (`chart_recommender.py`)

Phase 2 | Week 5–6

- **Technical Pair:** Works with **Adel Ahmed** and **Retaj Hakim**.
- **TASK-RE-04 (`backend/agent/answer_synthesizer.py`):** Build the synthesis module that transforms raw SQL result rows into natural Egyptian Arabic executive summaries (e.g., "إجمالي مبيعات الخمس منتجات المتصدرة بلغت 16.5 مليون دولار...").
- **TASK-RE-05 (`backend/agent/chart_recommender.py`):** Build the heuristic data-shape analyzer that inspects column data types and row cardinality to output structured `ChartConfig` JSON (`bar`, `pie`, `line`, `kpi`, or `table`) with Arabic axis titles for the Next.js frontend.
- **Deliverable:** Production `answer_synthesizer.py` and `chart_recommender.py` modules.

[TASK-RE-06 & RE-07] ~ Write Security Gate Unit Tests (`test_validator.py`) & Route Integration Tests — Phase 2–3 | Week 6–8

- **Action Plan:** Author $15+$ AST security tests in `backend/tests/test_validator.py` (verifying blocklist of `DROP`, `DELETE`, `UPDATE`, `EXEC xp_cmdshell`, and SQLi payloads) and mocked FastAPI route tests in `test_chat_route.py`.
- **Deliverable:** 100% passing backend security and API integration test suite.

[TASK-RE-08 to RE-10] ~ Adversarial Safety Audit, RTX 3050 Latency Benchmarks & QA Report — Phase 4–5 | Week 9–10

- **Action Plan:** Execute the 20-prompt adversarial safety audit (100% SC target) and 20-query end-to-end latency benchmark (< 3.0s target) into `benchmark_results.json`; author `Docs/QA-Report.md` and present the NLP & Security Benchmarks in Week 10.
- **Deliverable:** `benchmark_results.json`, `Docs/QA-Report.md`, and benchmark slides.

6. Retaj Hakim

Frontend Lead & Data Visualization Engineer (Next.js 14 / RTL)

Technical Ownership (10 Tasks): Architects the Next.js 14 TypeScript/Tailwind application with native RTL and Cairo typography, builds the 3-column layout, Header health monitor, Chat Bubbles, Egyptian Prompt Chips input bar, Recharts visualization suite (`Bar`, `Line`, `Pie`, `KPI`), sortable `DataTable` with CSV/Excel export, `SQLInspector` drawer, and frontend unit tests, plus authors the UI User Manual.

[TASK-R-01 & R-02] ~ Initialize Next.js 14 App Router, Native RTL Layout, Cairo Font & TypeScript Types — Phase 0 | Week 1–2

- **TASK-R-01:** Initialize Next.js 14 with TypeScript and Tailwind CSS; configure `<html lang="ar" dir="rtl">` and Google Font `Cairo` in `src/app/layout.tsx`.
- **TASK-R-02:** Define strict TypeScript contracts in `frontend/src/types/index.ts` (`ChatMessage`, `QueryData`, `ChartConfig`, `HealthStatus`).
- **Deliverable:** Next.js 14 RTL foundation and shared TypeScript interfaces.

[TASK-R-03 & R-04] ~ Build 3-Column Workspace (`page.tsx`), Live Header & Chat Bubbles — Phase 3 | Week 7–8

- **Technical Pair:** Works with **Ro'aa Ashraf** and **Moustafa Maher**.
- **TASK-R-03 (`page.tsx` & `Header.tsx`):** Build the responsive 3-column grid layout and header component polling `GET /api/v1/health` every 30s to display live SQL Server and GPU status.
- **TASK-R-04 (`UserBubble`, `AssistantBubble`, `ThinkingIndicator`):** Build RTL chat bubbles supporting embedded charts, tables, SQL toggles, and animated WebSocket thinking stages.
- **Deliverable:** Main 3-column interface, live health badge, and interactive chat thread.

[TASK-R-05 & R-06] ~ Build `ArabicInputBar` with Prompt Chips & Recharts Visualization Suite — **Phase 3 | Week 7–8**

- **Technical Pair:** Works with **Renad Hakim** (matching `chart_recommender.py` output schema).
- **TASK-R-05 (`ArabicInputBar.tsx`):** Build auto-resizing RTL input bar with Enter-to-send and 6 clickable Egyptian Arabic quick-prompt chips (e.g., [نواقص المخازن], [أكتر 5 منتجات مبيعاً]).
- **TASK-R-06 (`DataChart.tsx` & `KPICard.tsx`):** Build dynamic `recharts` wrapper rendering responsive Bar, Line, and Pie charts plus localized `Intl.NumberFormat('ar-EG')` KPI cards.
- **Deliverable:** `ArabicInputBar.tsx`, `DataChart.tsx`, and `KPICard.tsx`.

[TASK-R-07 & R-08] ~ Build Sortable `DataTable` (CSV/Excel Export), `SQLInspector` & Zustand Store — **Phase 3 | Week 7–8**

- **TASK-R-07 (`DataTable.tsx`):** Build paginated (10 rows/page), column-sortable RTL data table with one-click **CSV Export** (`papaparse`) and **Excel Export** (`xlsx`).
- **TASK-R-08 (`SQLInspector.tsx` & `store.ts`):** Build collapsible syntax-highlighted T-SQL drawer (`react-syntax-highlighter`) with copy-to-clipboard and retry attempt badges, backed by Zustand global state (`store.ts`).
- **Deliverable:** Complete data table with export, SQL inspector drawer, and Zustand store.

[TASK-R-09 & R-10] ~ Frontend Component Unit Tests (`Jest`) & Bilingual UI User Manual — **Phase 4–5 | Week 9–10**

- **TASK-R-09 (Pair with Renad):** Write React Testing Library unit tests for `ArabicInputBar.test.tsx`, `DataTable.test.tsx`, and `SQLInspector.test.tsx`.
- **TASK-R-10 (Shared Docs & Defense):** Author `Docs/User-Manual.md` with annotated RTL UI screenshots and present the Frontend & Data Visualization section in Week 10.
- **Deliverable:** Passing Jest component test suite and illustrated `User-Manual.md`.

5. Master Task Assignment Matrix (10 Tasks Per Engineer = 60 Total)

Every team member owns exactly **10 deliverables** combining core software engineering (80%) and shared testing/documentation (20

Task ID	Primary (Pair)	Phase	Technical Deliverable Description	Status
TASK-M-01	Moustafa	0	Initialize GitHub repo, branch protection & Kanban board	PENDING
TASK-M-02	Moustafa	0	Create root `.gitignore` & lock Python/training requirements	PENDING
TASK-M-03	Moustafa (Adel)	2	Build autonomous `self_correction.py` (3x reflection loop)	PENDING
TASK-M-04	Moustafa (Abdallah)	2	Implement `WS /api/v1/chat/ws` real-time streaming route	PENDING
TASK-M-05	Moustafa (Retaj)	3	Build typed Frontend REST API client (`src/lib/api.ts`)	PENDING
TASK-M-06	Moustafa (Retaj)	4	Build `useAgentStream.ts` WebSocket React hook	PENDING
TASK-M-07	Moustafa (All)	4	Wire end-to-end CORS, env config & full-stack integration	PENDING
TASK-M-08	Moustafa (All)	4	Lead PR code reviews, conflict resolution & release merge	PENDING
TASK-M-09	Moustafa	5	Author System Architecture in `README.md` & Design Doc	PENDING
TASK-M-10	Moustafa	5	Co-present Architecture & Self-Correction in final defense	PENDING
TASK-A-01	Adel	0	Configure CUDA 12.1 GPU env & Colab fallback notebook	PENDING
TASK-A-02	Adel	0	Download `Qwen2.5-Coder-3B` & log zero-shot SQL baseline	PENDING
TASK-A-03	Adel (Ro'aa)	1	Build gold T-SQL templates in `generate_synthetic_data.py`	PENDING
TASK-A-04	Adel	1	Implement 4-bit QLoRA fine-tuning script (`train_lora.py`)	PENDING
TASK-A-05	Adel	1	Merge LoRA weights & quantize to `Q4_K_M` GGUF format	PENDING
TASK-A-06	Adel (Moustafa)	2	Implement `model_runner.py` (`llama-cpp-python` GPU engine)	PENDING
TASK-A-07	Adel (Renad)	2	Implement ChatML `prompt_templates.py` (SQL, Reflect, Synth)	PENDING
TASK-A-08	Adel (Ro'aa)	4	Fine-tune weak domains to guarantee $\geq$ 85% EX accuracy	PENDING
TASK-A-09	Adel	5	Author QLoRA Training & Quantization section in Tech Docs	PENDING
TASK-A-10	Adel	5	Co-present AI Fine-Tuning & LLM Engine in final defense	PENDING

Task ID	Primary (Pair)	Phase	Technical Deliverable Description (Continued)	Status
TASK-RO-01	Ro'aa (Abdallah)	0	Write Python SQL schema extractor (`extract_schema.py`) → JSON	PENDING
TASK-RO-02	Ro'aa (Adel)	1	Build Egyptian dialect & DDL injector in `generate_synthetic_data.py`	PENDING
TASK-RO-03	Ro'aa (Adel)	1	Implement automated SQL evaluation script (`evaluate_model.py`)	PENDING
TASK-RO-04	Ro'aa (Abdallah)	2	Implement RAG Schema Pruner (`backend/agent/schema_loader.py`)	PENDING
TASK-RO-05	Ro'aa	2	Implement GET `/api/v1/schema` route & `test_schema_loader.py`	PENDING
TASK-RO-06	Ro'aa (Retaj)	3	Build Frontend `SchemaExplorer.tsx` interactive accordion sidebar	PENDING
TASK-RO-07	Ro'aa (Retaj)	3	Wire schema search filter & click-to-prompt hint integration	PENDING
TASK-RO-08	Ro'aa	4	Author `Docs/API-Reference.md` with REST/WS schemas	PENDING
TASK-RO-09	Ro'aa (All)	5	Record narrated 3–5 min end-to-end `demo-video.mp4`	PENDING
TASK-RO-10	Ro'aa	5	Co-present RAG Schema Pruner & Dataset Pipeline in defense	PENDING
TASK-AB-01	Abdallah	0	Initialize FastAPI `main.py`, CORS & Pydantic `config.py`	PENDING
TASK-AB-02	Abdallah (Ro'aa)	0	Restore `AdventureWorks2019` & build pyodbc `sql_executor.py`	PENDING
TASK-AB-03	Abdallah (Renad)	2	Implement `sqlglot` AST security validator (`sql_validator.py`)	PENDING
TASK-AB-04	Abdallah (Moustafa)	2	Implement `POST /api/v1/chat` full orchestration endpoint	PENDING
TASK-AB-05	Abdallah	2	Implement `GET /api/v1/health` DB/GPU diagnostics endpoint	PENDING
TASK-AB-06	Abdallah (Renad)	2	Write `test_executor.py` database execution unit tests	PENDING
TASK-AB-07	Abdallah	4	Implement LRU query caching & pyodbc auto-reconnect logic	PENDING
TASK-AB-08	Abdallah (Renad)	4	Optimize SQL query execution timeout & latency bottlenecks	PENDING
TASK-AB-09	Abdallah	5	Author Backend & AST Security section in Technical Design Doc	PENDING
TASK-AB-10	Abdallah	5	Co-present FastAPI Core & AST Safety Gate in final defense	PENDING
TASK-RE-01	Renad	0	Configure `pytest` (`conftest.py`) & Frontend `Jest` frameworks	PENDING
TASK-RE-02	Renad (Adel/Ro'aa)	1	Curate Egyptian dialect edge-cases & adversarial test prompts	PENDING
TASK-RE-03	Renad (Ro'aa)	2	Implement `arabic_normalizer.py` (Normalizer, Franco, Domains)	PENDING
TASK-RE-04	Renad (Adel)	2	Implement Egyptian Arabic `answer_synthesizer.py` module	PENDING

Task ID	Primary (Pair)	Phase	Technical Deliverable Description (Continued)	Status
TASK-RE-05	**Renad** (Retaj)	2	Implement automated `chart_recommender.py` analytics engine	PENDING
TASK-RE-06	**Renad** (Abdallah)	2	Write 15+ unit tests for `sql_validator.py` & `test_normalizer.py`	PENDING
TASK-RE-07	**Renad**	3	Write `/api/v1/chat` integration tests with mocked LLM runner	PENDING
TASK-RE-08	**Renad** (Abdallah)	4	Execute 20-prompt adversarial safety audit & latency benchmark	PENDING
TASK-RE-09	**Renad**	5	Compile `Docs/QA-Report.md` & `benchmark_results.json`	PENDING
TASK-RE-10	**Renad**	5	Co-present NLP Normalizer, Chart Engine & QA Metrics in defense	PENDING
TASK-R-01	**Retaj**	0	Initialize Next.js 14 App Router, Tailwind RTL & Cairo font	PENDING
TASK-R-02	**Retaj** (Moustafa)	0	Define shared TypeScript interfaces (`frontend/src/types/index.ts`)	PENDING
TASK-R-03	**Retaj** (Ro'aa)	3	Build 3-column responsive layout (`page.tsx`) & `Header.tsx`	PENDING
TASK-R-04	**Retaj**	3	Build `UserBubble`, `AssistantBubble` & `ThinkingIndicator`	PENDING
TASK-R-05	**Retaj**	3	Build `ArabicInputBar.tsx` with 6 Egyptian quick-prompt chips	PENDING
TASK-R-06	**Retaj** (Renad)	3	Build `DataChart.tsx` (Recharts Bar/Line/Pie) & `KPICard.tsx`	PENDING
TASK-R-07	**Retaj**	3	Build sortable `DataTable.tsx` with CSV & Excel export	PENDING
TASK-R-08	**Retaj**	3	Build syntax-highlighted `SQLInspector.tsx` & Zustand `store.ts`	PENDING
TASK-R-09	**Retaj** (Renad)	4	Write React component unit tests (Input, Table, SQLInspector)	PENDING
TASK-R-10	**Retaj**	5	Author `Docs/User-Manual.md` & co-present Frontend UI in defense	PENDING

6. Collaboration Governance & Risk Register

Engineering Collaboration & Pair Rotation Rules

1. **Pair Programming & Cross-Review:** Every module has a **Primary Owner** and a **Pair Partner** (shown in the Master Matrix). Partners review each other's PRs and can seamlessly swap tasks during exam weeks.
2. **Feature Branching:** Every task uses a dedicated branch (`feat/TASK-ID-short-description`). Direct pushes to `main` are blocked.
3. **Doc-As-You-Code Rule:** No single person is burdened with writing all documentation. Every engineer documents the exact API, algorithm, or UI component they coded before their PR is merged.
4. **Conventional Commits:** Follow structured commit prefixes: `feat(agent): add rag schema pruner`, `feat(nlp): add egyptian normalizer`.
5. **2-Hour Blocker Escalation:** Anyone blocked for > 2 hours immediately tags their Pair Partner on WhatsApp/Discord for a live pair-debugging session.

Project Risk Register & Mitigation Matrix

Identified Risk	Prob.	Impact	Squad Owners	Mitigation Strategy
Model EX $< 85\%$ after fine-tuning	Medium	**High**	**Adel** + **Ro'aa**	Expand synthetic samples in weak domains; increase LoRA rank to `r=32`.
RTX 3050 VRAM overflow during training	Medium	**High**	**Adel**	Offload training execution to prepared Google Colab T4 GPU notebook.
SQL Server connection drops in live demo	Low	**High**	**Abdallah** + **Moustafa**	Pre-cache 10 golden demo responses; implement automatic pyodbc connection retry logic.
Next.js RTL layout rendering glitches	Medium	**Medium**	**Retaj** + **Ro'aa**	Test across Chrome/Edge early; enforce explicit `dir="rtl"` inheritance at container levels.
Complex Egyptian slang / Franco queries	Medium	**Medium**	**Renad** + **Ro'aa**	Expand `arabic_normalizer.py` dictionary and synthetic prompt templates.
Team member unavailability (exams)	Medium	**Medium**	**All Pairs**	Because every task has a Pair Partner, the partner can immediately take over open branches.